

Predicting Product Success in Sports Footwear Retail: A Multi-Model Classification Approach

DS675 Machine Learning — Mini-Project Milestone 4

Manuj Arora, Vivek Gupta
New Jersey Institute of Technology
Spring 2026

Dataset: [Sports Footwear Sales and Consumer Behavior \(Kaggle\)](#)

Code: [GitHub Code Repository](#)

Video Presentation: [Recording](#)

PowerPoint presentation: [Presentation](#)

Milestone 2 Report: [Report](#)

1. Introduction and Objectives

The global sports footwear market generates billions in annual revenue, yet retailers lack data-driven tools for a critical decision: before stocking a new shoe, can they predict whether it will be a Hit, Average, or Flop — based on brand, category, price, discount, sales channel, and market? This project builds a supervised ML system that classifies products into three success tiers — High Performing, Average, and Underperforming — using only attributes available before a sale occurs. While demonstrated on footwear, the framework applies to retail broadly.

We compare three supervised models — Logistic Regression, XGBoost, and a PyTorch MLP — supplemented by K-Means clustering to assess whether success structure emerges naturally, and PCA for dimensionality reduction.

2. Dataset Description

The Sports Footwear Sales and Consumer Behavior dataset, sourced from Kaggle, contains 30,000 retail transactions spanning 2018 to 2026. It covers six major global brands (Nike, Adidas, Puma, Reebok, ASICS, New Balance) across five product categories (Running, Lifestyle, Basketball, Training, Gym), three gender segments, and six international markets (USA, UK, India, Germany, Pakistan, UAE). The dataset has a usability score of 10.0 with zero missing values across all 18 features.

Table 1: Dataset Feature Summary

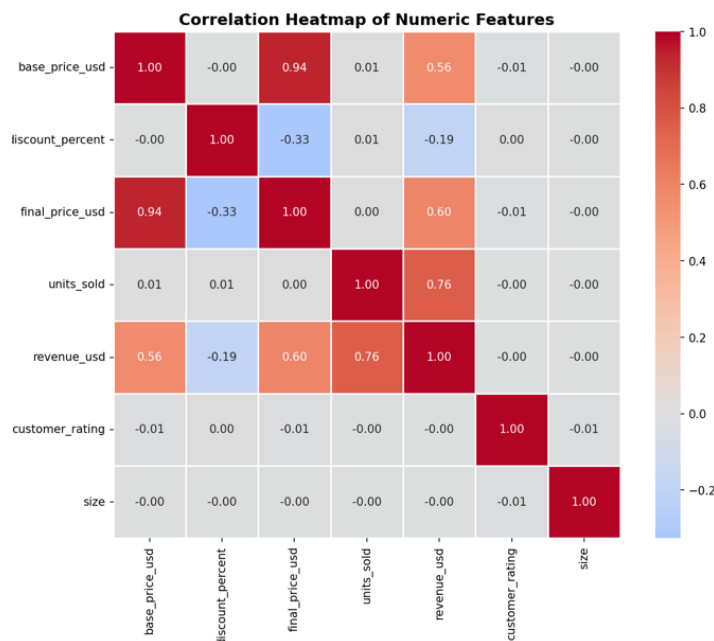
Feature(s)	Type	Description / Role
brand, category, gender, color	Categorical	Product attributes — one-hot encoded as input features
base_price_usd, discount_percent	Numeric	Pricing attributes — also used to derive interaction feature
units_sold, size	Numeric	Transaction and product attributes
sales_channel, payment_method	Categorical	Purchase context (Online vs Retail Store, Cash/Card/Wallet)
country, customer_income_level	Categorical	Market and demographic attributes
order_date	Date	Decomposed into year, month, quarter temporal features
revenue_usd, customer_rating	Numeric	Used ONLY to construct target label, then removed (see Section 3)
final_price_usd	Numeric	Dropped — deterministic from $\text{base_price} \times (1 - \text{discount}/100)$
order_id, model_name	Identifier	Dropped — not predictive (order_id) or too high cardinality (899 unique)

2.1 Exploratory Data Analysis

Table 2: Summary Statistics of Key Numeric Features

Feature	Mean	Std Dev	Min	Median	Max
base_price_usd	\$139.63	\$46.06	\$60	\$140	\$219
discount_percent	13.33%	9.86%	0%	10%	30%
units_sold	2.50	1.12	1	3	4
revenue_usd	\$302.71	\$179.15	\$42	\$268	\$876
customer_rating	4.00	0.58	3.0	4.0	5.0

Correlation analysis revealed that `base_price_usd` and `final_price_usd` are highly correlated ($r = 0.940$), confirming the redundancy that justifies dropping `final_price_usd`. `Units_sold` and `revenue_usd` show strong correlation ($r = 0.760$), as expected. Categorical features are approximately uniformly distributed, with each brand appearing in roughly 5,000 transactions.



3. Data Preparation and Feature Engineering

3.1 Target Variable Engineering

The target combines `revenue_usd` (business performance) and `customer_rating` (satisfaction). Neither alone is sufficient — a product might generate high revenue through discounting but receive poor ratings. Construction proceeds in three steps:

1. **Step 1 — Normalize:** Z-score both `revenue_usd` and `customer_rating` to place them on a common scale (mean = 0, standard deviation = 1). This prevents revenue (range: ~\$100–\$800) from dominating customer_rating (range: 1–5).
2. **Step 2 — Combine:** Average the two z-scores into a single `success_score` = (`revenue_z` + `rating_z`) / 2.
3. **Step 3 — Discretize:** Bin into three classes using quantile-based thresholds at the 33rd and 67th percentiles, producing balanced classes: Underperforming (10,000), Average (10,000), and High Performing (10,000).

3.2 Preventing Data Leakage

Data leakage is one of the most critical pitfalls in machine learning. It occurs in two distinct forms, and our project implements safeguards against both.

- Type 1 — Target Leakage occurs when training features contain information unavailable at prediction time. If `revenue_usd` and `customer_rating` were kept as inputs, the model would trivially reverse-engineer the label, achieving near-perfect accuracy that would be useless in practice — a retailer predicting success for an unsold shoe has neither metric available.
- Type 2 — Train-Test Contamination occurs when training set information leaks into evaluation sets. For example, fitting `StandardScaler` on the entire dataset before splitting would encode test data properties into the scaler, producing biased metrics.

We implement a strict four-point prevention protocol against both types:

- **Removal of target-derived features (Type 1):** Both `revenue_usd` and `customer_rating` are permanently removed from the feature set after target labels are created. The model never sees these values during training, validation, or testing.

- **Removal of deterministic features (Type 1):** `final_price_usd` is a deterministic function of `base_price_usd` and `discount_percent` ($\text{final_price} = \text{base_price} \times (1 - \text{discount}/100)$). Retaining it would introduce redundancy and a potential leakage pathway since it is directly correlated with `revenue_usd`. It is dropped.
- **Prediction-time validity (Type 1):** The remaining 16 features (which become 34 after encoding) are all attributes that a retailer would know before a product is sold: brand, category, gender, price, planned discount, sales channel, country, customer income level, and time of year. No historical sales data is required at inference time.
- **Preprocessing isolation (Type 2):** The `StandardScaler` and `OneHotEncoder` are fit exclusively on the training set. Validation and test sets are transformed using the training-set statistics only, ensuring that no information from the evaluation sets leaks into the preprocessing step. This guarantees that test set performance reflects true out-of-sample generalization.

3.3 Feature Engineering

We engineer the following new features:

- **Temporal features:** We extract year, month, and quarter from `order_date`. This allows models to capture seasonal purchasing patterns (e.g., holiday-season performance) and year-over-year trends. Neither Kaggle author performed this extraction.
- **Interaction feature:** We create `price_sensitivity = base_price_usd × discount_percent`. A 20% discount on a \$200 shoe (\$40 off) has a fundamentally different market impact than the same percentage on a \$60 shoe (\$12 off). Neither `base_price` nor `discount` alone captures this dollar-value interaction.
- **Categorical encoding:** Seven categorical features are one-hot encoded with the first category dropped to avoid multicollinearity. This expands the feature space from 16 raw columns to 34 encoded dimensions.
- **Numeric scaling:** All numeric features are standardized using `StandardScaler` (zero mean, unit variance), which is essential for distance-based methods (PCA) and gradient-based optimization (MLP).

3.4 Data Partitioning

The dataset is split into three stratified subsets to ensure class proportions (33.3% each) are maintained:

Table 3: Data Partition

Partition	Samples	Proportion	Purpose
Training	21,000	70%	Model training and cross-validation
Validation	4,500	15%	Hyperparameter tuning and model selection
Test	4,500	15%	Final unbiased evaluation (never seen during training)

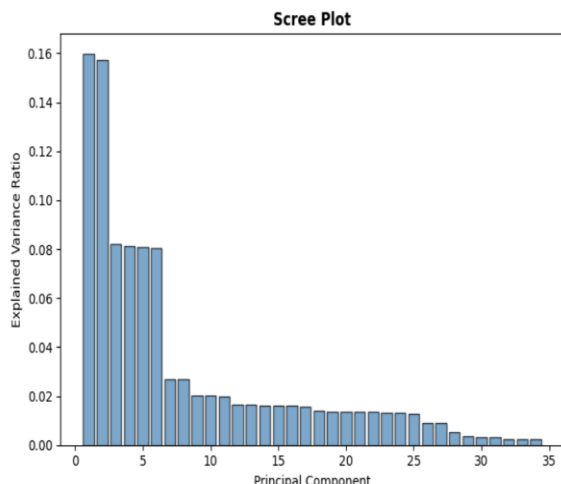
4. Dimensionality Reduction — PCA

Principal Component Analysis is applied to the 34-dimensional processed feature space to understand the data's underlying structure before modeling.

Variance Analysis: We compute all 34 principal components and examine the proportion of total variance each captures. The analysis reveals that 21 components are needed to retain 90% of the cumulative variance and 25 components for 95%. This indicates that the feature space is not highly redundant — most encoded dimensions carry meaningful information, and aggressive dimensionality reduction would risk losing predictive signal. This finding justifies our decision to retain all 34 features as inputs to the supervised models rather than projecting onto a reduced subspace.

2D Visualization: We project the training data onto the first two principal components to visually assess class separability. The resulting scatter plot — colored by product success class — reveals significant overlap among the three classes, with no distinct clusters forming. This overlap confirms that the classification problem is non-trivial and cannot be solved by simple linear separation in the original feature space.

Linear PCA vs Kernel PCA (RBF): To test whether non-linear transformations reveal better structure, we also apply Kernel PCA with a Radial Basis Function (RBF) kernel. The RBF kernel maps data into a higher-dimensional space where non-linear relationships may become linearly separable. However, the Kernel PCA projection shows only marginal improvement in class separation compared to linear PCA. This suggests that the class boundaries are driven by complex interactions across many features rather than a single non-linear manifold — further motivating the use of ensemble methods (XGBoost) and neural networks (MLP) that can capture these multi-dimensional interactions.



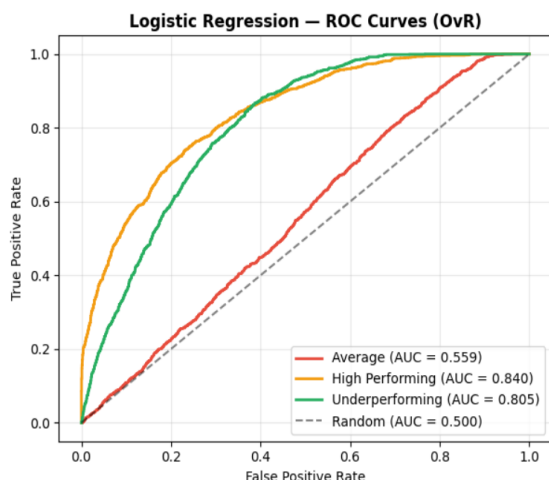
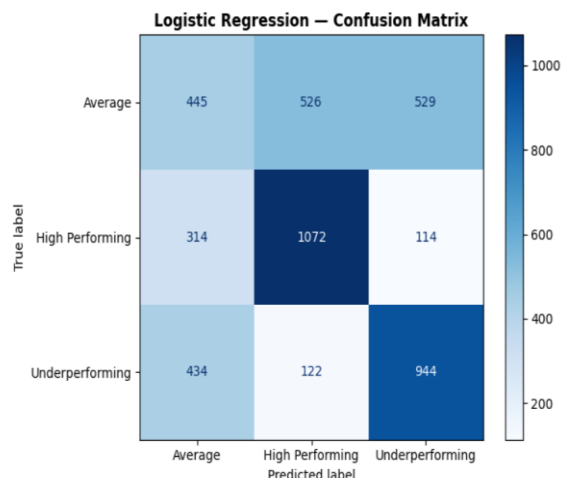
5. Supervised Classification Models

5.1 Logistic Regression

Logistic Regression serves as our linear baseline — the simplest supervised classifier in our pipeline. It models the probability of each class using a softmax function applied to a linear combination of input features. Despite its simplicity, it provides a critical reference point: if more complex models cannot meaningfully outperform Logistic Regression, it suggests that the performance ceiling is determined by the feature space rather than model architecture.

We apply L2 regularization to prevent overfitting by penalizing large weight values. The regularization strength is controlled by the hyperparameter C, which is the inverse of the penalty — smaller C means stronger regularization. We tune C via Grid Search with 5-fold stratified cross-validation over the range [0.001, 0.01, 0.1, 1, 10, 100]. The best configuration (C = 0.1) achieved a cross-validation Macro F1 of 0.536, indicating that moderate regularization is optimal — the model benefits from some constraint on its weights without being overly restricted.

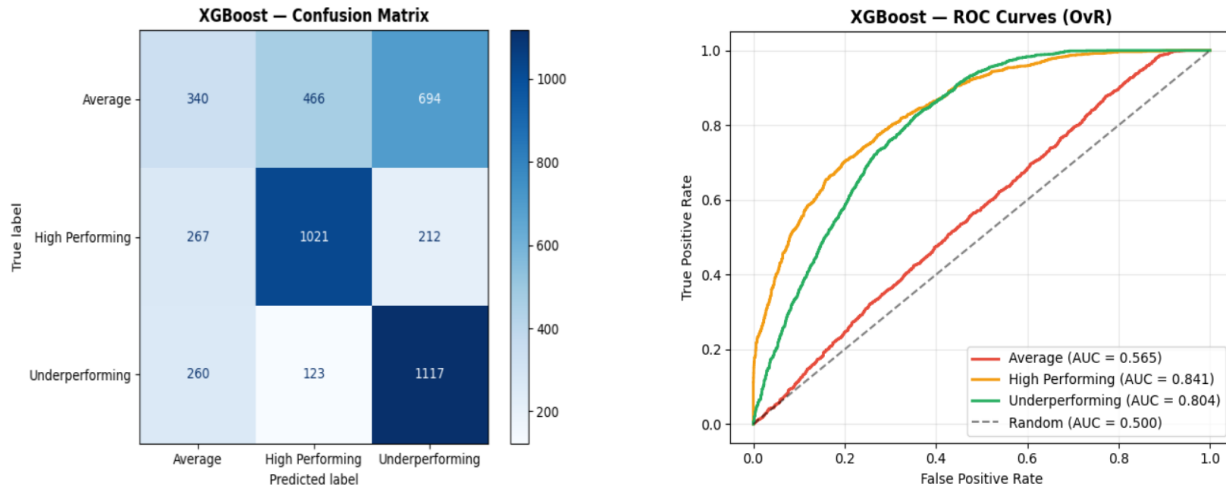
This model directly mirrors the Kaggle authors' use of Linear Regression, adapted for our classification task. Where they predicted a continuous revenue value, we predict a discrete success category using the same underlying linear framework.



5.2 XGBoost — Gradient Boosted Trees

XGBoost is a sequential ensemble method where decision trees are built one after another, with each new tree specifically targeting the errors made by the previous trees. Unlike Random Forest, which builds trees independently and averages them, XGBoost's sequential correction mechanism allows it to progressively reduce both bias and variance — making it one of the most powerful algorithms for structured tabular data.

We perform Grid Search over 54 parameter combinations: `n_estimators` ∈ {100, 300, 500} controls how many trees are built, `max_depth` ∈ {4, 6, 8} limits each tree's complexity to prevent overfitting, `learning_rate` ∈ {0.01, 0.05, 0.1} controls how much each tree contributes to the final prediction, and `subsample` ∈ {0.8, 1.0} determines the fraction of training data each tree sees. The best configuration (`learning_rate`=0.05, `max_depth`=4, `n_estimators`=300, `subsample`=0.8) achieved a cross-validation Macro F1 of 0.526.



Feature importance analysis from the tuned model reveals which attributes are most predictive of product success, providing business-interpretable insights. This also serves as our direct benchmark against Ali Hussain's Kaggle XGBoost regressor — same algorithm, different task.

Table 4: Top 10 XGBoost Feature Importances

Rank	Feature	Business Interpretation
1	units_sold	Volume is the strongest driver of the composite success score
2	base_price_usd	Higher-priced products tend toward higher revenue and thus higher success
3	price_sensitivity	The dollar impact of discounting matters more than discount % alone
4	discount_percent	Discount level influences both revenue and consumer perception
5	size	Certain sizes may correlate with popular models or demographics
6	order_month	Seasonal patterns in purchasing behavior
7	order_year	Year-over-year market trends and brand evolution
8–10	brand, category, country indicators	Categorical attributes contribute modestly after encoding

The engineered `price_sensitivity` feature ranks third, validating our feature engineering. Categorical features contribute individually less but collectively represent a meaningful signal.

5.3 PyTorch MLP — Multi-Layer Perceptron

The neural network is our key differentiator — neither Kaggle author attempted deep learning on this dataset. We implement a fully connected feedforward network (MLP) in PyTorch, where every neuron in one layer connects to every neuron in the next, and data flows in one direction from input to output.

The architecture progressively compresses the representation: Input(34) → Hidden1(128, ReLU, BatchNorm, Dropout 0.3) → Hidden2(64, ReLU, BatchNorm, Dropout 0.3) → Hidden3(32, ReLU, BatchNorm) → Output(3). Each hidden layer learns increasingly abstract feature combinations — the first layer captures basic patterns

across the 34 inputs, the second combines these into higher-level representations, and the third distills them into the most discriminative features for the three-class prediction.

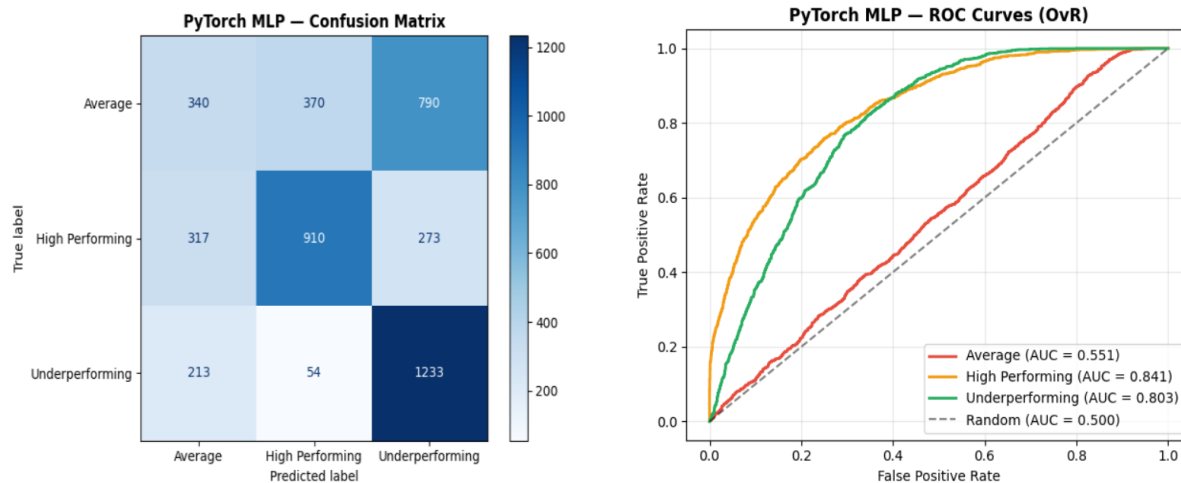
The model applies several key concepts from the course: ReLU activation is used in hidden layers instead of sigmoid or tanh to avoid the vanishing gradient problem, where gradients shrink to near-zero in deep networks and early layers stop learning. CrossEntropyLoss combines LogSoftmax and negative log-likelihood — minimizing this loss is mathematically equivalent to maximizing the likelihood of the correct class labels (MLE). PyTorch's autograd engine handles backpropagation automatically through the 5-step training loop: forward pass → compute loss → zero gradients → backward pass → optimizer step. Regularization is applied through three mechanisms: Dropout randomly deactivates 30% of neurons during training to prevent co-adaptation, weight_decay adds an L2 penalty to the optimizer, and early stopping (patience=15) halts training when validation performance stops improving.

Training was conducted with the Adam optimizer (lr=0.001) with ReduceLROnPlateau scheduling, which automatically reduces the learning rate when validation loss plateaus. Early stopping triggered at epoch 20, yielding a best validation Macro F1 of 0.5263.

Table 5: MLP Architecture Comparison

Architecture	Parameters	Dropout	Best Val F1
Shallow (64)	2,563	0.3	0.5314
Medium (128 → 64)	13,315	0.3	0.5283
Deep (128 → 64 → 32)	15,363	0.3	0.5200
Wide (256 → 128 → 64)	51,203	0.3	0.5220
Deep + Low Dropout (0.1)	15,363	0.1	0.5247
Deep + High Dropout (0.5)	15,363	0.5	0.5371

The best variant was Deep + High Dropout (0.5) at F1 = 0.5371. Notably, the Shallow (64) network with just 2,563 parameters achieved competitive F1 = 0.5314, while the Wide variant (51,203 params) performed worse (0.5220) — confirming that added capacity leads to overfitting on this tabular dataset.



6. Unsupervised Comparison — K-Means

K-Means (k=3) clusters the data without labels to test whether success tiers emerge naturally. Result: ARI = 0.0000, Silhouette Score = 0.1072. The clusters (sizes: 7,867, 7,760, 5,373) show zero correspondence with actual labels — demonstrating that supervised learning is essential for this task and validating our classification approach.

7. Hyperparameter Tuning Summary

All models are tuned using Grid Search with 5-fold stratified cross-validation. The table below consolidates each model's search space and best configuration.

Table 7: Hyperparameter Tuning Details

Model	Hyperparameter	Search Space	Best Value
Logistic Regression	C (regularization strength)	{0.001, 0.01, 0.1, 1, 10, 100}	0.1
Logistic Regression	solver	{lbfgs}	lbfgs
XGBoost	n_estimators	{100, 300, 500}	300
XGBoost	max_depth	{4, 6, 8}	4
XGBoost	learning_rate	{0.01, 0.05, 0.1}	0.05
XGBoost	subsample	{0.8, 1.0}	0.8
PyTorch MLP	architecture	6 variants (see Table 5)	Deep 128→64→32
PyTorch MLP	dropout_rate	{0.1, 0.3, 0.5}	0.3 (main) / 0.5 (best)
PyTorch MLP	learning_rate	{0.001}	0.001
PyTorch MLP	weight_decay (L2)	{1e-4}	1e-4
PyTorch MLP	early stopping patience	{15 epochs}	Triggered at epoch 20
K-Means	n_clusters (k)	{3} (fixed to match classes)	3
K-Means	n_init (runs)	{10}	10
K-Means	init method	{k-means++}	k-means++

8. Results and Model Comparison

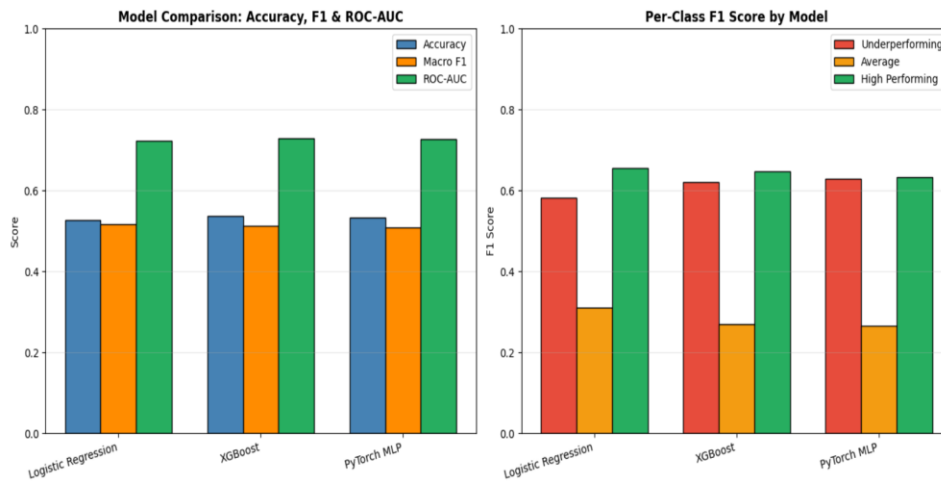
All models are evaluated on the validation set (4,500 samples) and held-out test set (4,500 samples) using Accuracy, Macro F1, and ROC-AUC (One-vs-Rest).

Table 6: Validation and Test Set Performance Comparison

Model	Accuracy	Macro F1	ROC-AUC	F1 Avg.	F1 High	F1 Under.
Logistic Regression (validation)	0.5469	0.536	0.7348	0.3305	0.6658	0.6116
XGBoost (validation)	0.5507	0.526	0.7363	0.2873	0.6566	0.6341
Pytorch MLP (validation)	0.5518	0.5263	0.7315	0.2869	0.6422	0.6496
Logistic Regression (test)	0.5267	0.5163	0.7238	0.3103	0.6554	0.5833
XGBoost (test)	0.538	0.5127	0.7293	0.2698	0.6468	0.6213
PyTorch MLP (test)	0.5338	0.5098	0.7268	0.2663	0.6338	0.6293

8.1 Key Observations

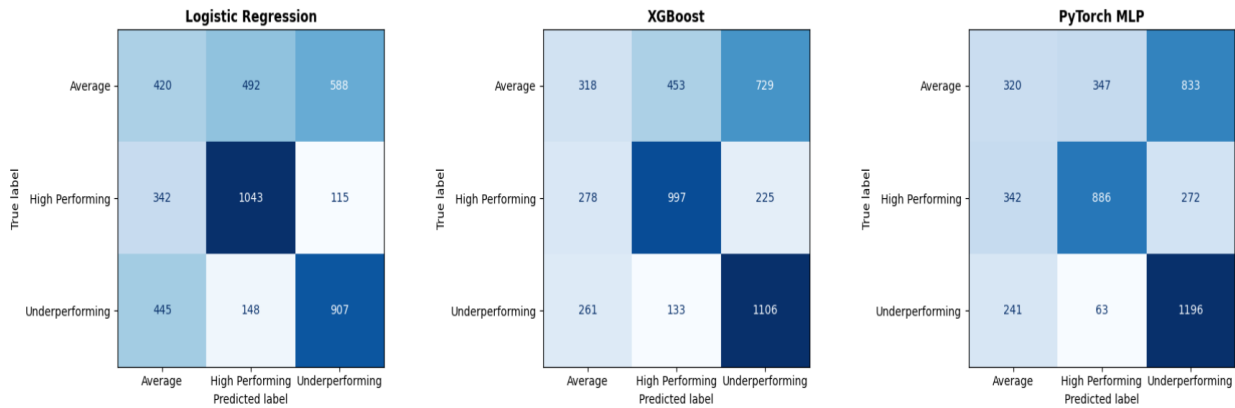
- **All models perform comparably:** Accuracies range from 52.67% to 53.8% and Macro F1 from 0.5098 to 0.5163. This narrow spread suggests that the performance ceiling is determined more by the feature space than the model architecture.
- **Significantly above random:** With three balanced classes, random guessing would yield ~33.3% accuracy and ~0.333 Macro F1. All models exceed this by 20+ percentage points, confirming that meaningful patterns exist in the data.
- **The Average class is hardest to predict:** Across all models, F1 for the Average class (0.266–0.310) is substantially lower than for Underperforming (0.58–0.63) and High Performing (0.63–0.66). Products at the extremes of the spectrum of success have more distinctive feature profiles, while middling products are harder to differentiate.
- **XGBoost and MLP did not surpass Logistic Regression in Macro F1:** On tabular data with 34 features and relatively weak feature-target correlations, the added model complexity does not translate to meaningful gains. This is a well-documented finding in the ML literature for structured/tabular datasets.



8.2 Confusion Matrix Analysis

All models show systematic patterns: the Underperforming and High Performing classes achieve reasonable F1 (0.58–0.66), while the Average class is the clear bottleneck with F1 of just 0.27–0.31. Models only correctly identify 21–28% of Average products — the remainder split between the two extremes. This is expected: Average products share feature characteristics with both neighbors, making class boundary discrimination inherently difficult and explaining why all models plateau at similar accuracy.

Confusion Matrices — Test Set



9. Retailer Showcase: Model Predictions on unseen data

To demonstrate practical applicability, we pass 16 unseen test products through the trained PyTorch MLP and display the model's predictions alongside actual outcomes. This simulates what a retailer would see when evaluating new products for stocking decisions — only product attributes are provided, and the model returns a predicted success tier with a confidence score.

The sample results show the model correctly classified 9 out of 16 products (56.3%). Notably, the model performs strongest when confidence is high — for instance, the New Balance Training shoe at \$190 with no discount was predicted High Performing with 87.0% confidence and was indeed correct. Conversely, most misclassifications occur when confidence is low (35–46%), indicating the model appropriately signals uncertainty on ambiguous products. The Average class accounts for the majority of errors, consistent with our confusion matrix findings — products in the middle tier lack the distinctive feature signatures that characterize clear hits or flops.

From a business perspective, a retailer could use these confidence scores as a decision filter: stock products where the model predicts High Performing with confidence above 60%, flag Average predictions for manual review, and deprioritize products predicted as Underperforming.

Generating retailer showcase predictions...										
=====										
WHAT THE RETAILER SEES: Sample Predictions (Pytorch Mlp)										
=====										
brand	category	gender	base_price_usd	discount_percent	sales_channel	country	Predicted	Confidence	Actual	Correct
ASICS	Running	Men	80	30	Retail Store	USA	Underperforming	40.6%	Underperforming	True
ASICS	Running	Unisex	78	10	Online	Germany	Underperforming	35.4%	Average	False
New Balance	Lifestyle	Unisex	104	0	Retail Store	Pakistan	Underperforming	49.8%	Underperforming	True
Puma	Lifestyle	Men	75	30	Online	UK	Underperforming	43.9%	Average	False
New Balance	Training	Unisex	190	0	Online	Pakistan	High Performing	87.0%	High Performing	True
Nike	Training	Men	73	10	Online	India	Underperforming	39.1%	Underperforming	True
Puma	Basketball	Unisex	121	5	Retail Store	USA	High Performing	59.0%	High Performing	True
New Balance	Gym	Unisex	179	30	Online	USA	Underperforming	67.5%	Underperforming	True
Reebok	Running	Men	71	5	Online	UAE	Underperforming	53.1%	Average	False
Adidas	Running	Men	115	30	Online	UK	Underperforming	46.5%	High Performing	False
New Balance	Running	Women	136	30	Online	Germany	Underperforming	39.2%	Underperforming	True
Reebok	Basketball	Unisex	142	30	Online	Germany	Underperforming	59.7%	Underperforming	True
ASICS	Running	Men	101	0	Online	UK	Underperforming	61.9%	Average	False
ASICS	Running	Women	207	10	Online	USA	High Performing	76.3%	Average	False
New Balance	Lifestyle	Men	133	5	Retail Store	USA	Underperforming	55.9%	Underperforming	True

10. Comparison with Existing Kaggle Work

The Kaggle community has engaged with this dataset through five contributions. Hussain developed an XGBoost Regressor for revenue prediction ($R^2 = 0.89$, $MAE \approx \$142.50$) using a single model with no cross-validation. Risk built a linear regression model ($R^2 \approx 0.93$) emphasizing visualization over model sophistication. Aydost and Shaikh each produced descriptive EDA notebooks with no predictive modeling. Chagas attempted a Random Forest binary classifier for order volume prediction but achieved only 49.9% accuracy — equivalent to random guessing.

Table 7: Comparison with Kaggle Contributions

Dimension	Hussain	Risk	Aydost / Shaikh	Chagas	Our Project
Task	Regression	Regression	EDA only	Binary classif.	3-Class classif.
Models	XGBoost	Linear Reg	None	Random Forest	3 sup. + K-Means
Deep Learning	No	No	No	No	PyTorch MLP
Feature Eng.	Minimal	Minimal	None	Minimal	Temporal + Inter.
Validation	Single split	Single split	N/A	Single split	Strat. CV + hold
Performance	$R^2=0.89$	$R^2\approx 0.93$	N/A	Acc=49.9%	Acc=53.8%

Our primary contribution is reframing from descriptive regression to actionable classification. Of five Kaggle contributions, three are purely descriptive, one did regression, and the one classification attempt achieved random-chance accuracy. Ours is the first meaningful classification on this dataset, with a neural network, multi-model comparison, and rigorous evaluation.

11. Conclusions and Limitations

All three supervised models achieve test accuracy around 53% and ROC-AUC above 0.72, significantly exceeding the 33.3% random baseline. Logistic Regression achieved the highest Macro F1 (0.5163), followed by XGBoost (0.5127) and MLP (0.5098). K-Means (ARI = 0.0) confirmed supervised learning is essential. The finding that complex models did not outperform the linear baseline indicates that on tabular retail data, model complexity is not the bottleneck — feature quality is. Future work could explore richer feature engineering, alternative target construction, or external data sources.

12. Contributions

Table 9: Team Member Contributions

Team Member	Contribution
Manuj Arora	Introduction and Objectives, Data Preparation and Feature Engineering Logistic Regression implementation and tuning Pytorch MLP implementation, training and architecture experiments PCA/Kernel PCA Analysis
Vivek Gupta	Exploratory Data Analysis and Visualizations XGBosst Implementation, Tuning and feature importance analysis K-Means Clustering Comparison Result Analysis, Confusion Matrix Interpretation, Kaggle Comparison

13. References

- **Dataset:** Sports Footwear Sales and Consumer Behavior — Kaggle
- **Kaggle Notebook 1:** Ali Hussain — Advanced Sales Forecasting (XGBoost Regressor, $R^2 = 0.89$)
- **Kaggle Notebook 2:** Brian Risk — Sales Analysis Prediction (Linear Regression, $R^2 \approx 0.93$)
- **Kaggle Notebook 3:** Fatih Aydost — Footwear Market: Interactive EDA (Plotly visualizations, no ML)
- **Kaggle Notebook 4:** Maaz Shaikh — Sports Footwear Sales and Consumer Behavior (descriptive analysis)
- **Kaggle Notebook 5:** Jadson Chagas — Sport Footwear Sales Consumer Behavior (Random Forest binary classifier, 49.9% accuracy)
- **Course Materials:** DS675 Machine Learning, NJIT, Spring 2026 — Modules 5, 6, 9, 11, 12, 13, 14, 15, 16, 18